MODULE

13

Errors, context managers & logging

Production code fails safely, cleans up after itself, and tells you what happened.

YOU WILL LEARN

Exceptions like a pro	01_exception_hierarchy.py
Context managers	02_context_managers.py
logging	03_logging.py
Demo: robust order importer	04_demo_order_importer.py
ExitStack & exception groups	05_exitstack_and_groups.py
Debugging like a pro	06_debugging_like_a_pro.py
Interview Q&A	INTERVIEW.md

WHY EXCEPTIONS

Errors are messages, not disasters

THE PROBLEM

A payment API times out. Bad code **crashes** the whole import, or worse, catches everything and **silently** continues with wrong data.

Good code decides: **retry, skip this row, roll back, or stop**.

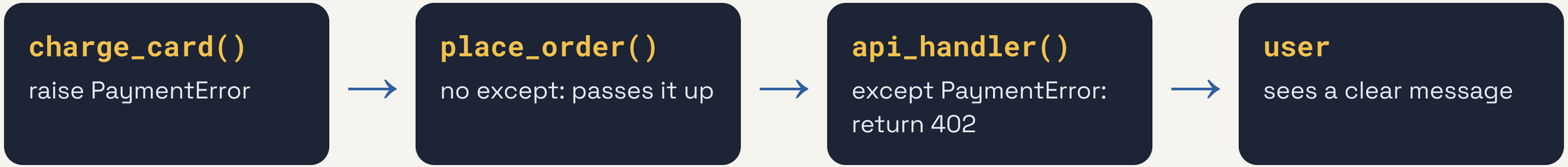
THE IDEA

Raise specific exceptions, catch only what you can handle, and let the rest go up to someone who can.

PICTURE IT EXCEPTIONS

What happens when an error is raised

🏀 An exception is like a ball thrown **up the call stack**. Each function can catch it (**except**) or let it fly. If nobody catches it, the program stops and prints a **traceback**.



TRY IT EXCEPTIONS

try / except / else / finally

```
def parse_amount(text):
    try:
        value = float(text)
    except ValueError:
        print(f"  {text!r} is not a number")
        return None
    else:                          # only if NO exception
        print(f"  parsed {value}")
        return value
    finally:                       # ALWAYS, even after return
        print("  (cleanup)")

parse_amount("19.99")
parse_amount("abc")
```

OUTPUT

```
  parsed 19.99
  (cleanup)
  'abc' is not a number
  (cleanup)
```

1. **except**: handle a specific error.
2. **else**: the success path, kept out of the try block.
3. **finally**: always runs; good for cleanup (but prefer **with**).

TRY IT EXCEPTIONS

Your own exception family

```python
class ShopError(Exception):
    """Base for everything our app raises."""

class OutOfStock(ShopError):
    def __init__(self, item, wanted, left):
        super().__init__(f"{item}: wanted {wanted}, only {left} left")
        self.item = item

class PaymentFailed(ShopError): ...

def checkout(item, qty):
    try:
        raise ConnectionError("gateway timeout")   # low-level
    except ConnectionError as e:
        raise PaymentFailed("card not charged") from e

try:
    checkout("lamp", 1)
except ShopError as e:              # catch the whole family
    print(type(e).__name__, "-", e)
    print("cause:", repr(e.__cause__))

print(OutOfStock("lamp", 3, 1))
```

OUTPUT

```
PaymentFailed - card not charged
cause: ConnectionError('gateway timeout')
lamp: wanted 3, only 1 left
```

1. One **base class** lets callers catch all your errors at once.
2. **raise ... from e** keeps the original cause in the traceback.
3. Add useful fields (**item**) for the handler.

COMMON MISTAKE EXCEPTIONS

Swallowing every error

✗ WRONG

```
try:
    total = compute_invoice(order)
    send_email(order.customer, total)
except:                          # catches EVERYTHING
    pass                         # ...and hides it
```

A bare **except** also catches typos (NameError), Ctrl+C and real bugs. The invoice silently isn't sent and nobody knows why.

✓ RIGHT

```
try:
    total = compute_invoice(order)
except InvalidOrder as e:
    log.warning("skipping order %s: %s", order.id, e)
    return
send_email(order.customer, total)   # errors here surface
```

Catch the **narrowest** exception you can handle, **log** it, and keep the try block small.

PICTURE IT CONTEXT MANAGERS

with = guaranteed clean-up

🚪 Like a **door that closes itself**: whatever happens inside the room, even an exception, the door is closed when you leave. Files, DB transactions, locks and HTTP sessions all work this way.

`with open(p) as f:`
`__enter__`: open, return f

→

`your code`
may succeed or raise

→

`__exit__`
ALWAYS runs: close the file

→

`exception?`
is passed on (unless handled)

TRY IT CONTEXT MANAGERS

Write one with @contextmanager

```python
from contextlib import contextmanager

@contextmanager
def transaction(db):
    try:
        yield db
        db.append("COMMIT")
    except Exception:
        db.append("ROLLBACK")
        raise

log = []
with transaction(log) as db:
    db.append("INSERT 1")
print(log)

log = []
try:
    with transaction(log) as db:
        db.append("INSERT 1")
        raise ValueError("row 2 is broken")
except ValueError as e:
    print("error:", e)
print(log)
```

OUTPUT

```
['INSERT 1', 'COMMIT']
error: row 2 is broken
['INSERT 1', 'ROLLBACK']
```

1. Code before **yield** = setup; after = teardown.
2. On an error: **roll back**, then re-raise.
3. Same idea as Django's **transaction.atomic()**.

COMMON MISTAKE LOGGING

print() is not logging

✗ WRONG

```
print("user logged in")
print("ERROR: payment failed", e)
```

No time, no level, no module name, can't be switched off, can't be sent to a log service. In production you're blind.

✓ RIGHT

```
import logging
log = logging.getLogger(__name__)

log.info("user %s logged in", user_id)
log.exception("payment failed for order %s", oid)
```

Levels, timestamps, module names, filtering and handlers. **log.exception** adds the full traceback.

TRY IT LOGGING

Configure once, log everywhere

```python
import logging, sys

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s %(levelname)-7s %(name)s: %(message)s",
    datefmt="%H:%M:%S", stream=sys.stdout)

log = logging.getLogger("shop.orders")
log.debug("cart contents: ...")          # below INFO: hidden
log.info("order %s placed", 1042)
log.warning("stock low for %s", "lamp")
try:
    1 / 0
except ZeroDivisionError:
    log.error("could not compute discount")
```

OUTPUT

```
12:01:07 INFO    shop.orders: order 1042 placed
12:01:07 WARNING shop.orders: stock low for lamp
12:01:07 ERROR   shop.orders: could not compute discount
```

1. Levels: **DEBUG < INFO < WARNING < ERROR < CRITICAL**.
2. Use **%s** arguments, not f-strings: formatting is skipped when the level is off.
3. One logger per module: **getLogger(__name__)**.

STEP BY STEP DEBUGGING

Debugging like a pro

TOOL	USE IT WHEN	HOW
`breakpoint()`	you want to stop and look around	n next · s step in · c continue · p x print · q quit
`post-mortem`	it crashed: inspect where	`python -m pdb script.py`
`VS Code debugger`	web apps and tests	launch.json for uvicorn / pytest, click to set breakpoints
`log.exception()`	it fails in production	full traceback in the logs
`faulthandler`	it hangs or segfaults	`python -X faulthandler app.py`

Read tracebacks **bottom-up**: the last line is the error, the line above is where it happened.

In real web services

Map domain errors to HTTP

```
@app.exception_handler(OutOfStock)
def out_of_stock(req, exc):
    return JSONResponse({"detail": str(exc)}, 409)
```

Atomic DB writes

```
with transaction.atomic():      # Django
    order.save(); stock.save()
```

Structured logs for search

```
log.info("order placed", extra={"order_id": 1042})
```

Never leak internals

```
except Exception:
    log.exception("unexpected")
    raise HTTPException(500, "Internal error")
```

INTERVIEW SAY THIS IN THE INTERVIEW

“How do you handle exceptions in a production API?”

- Define a **custom exception hierarchy** for business errors (OutOfStock, PaymentFailed).
- Catch **specific** exceptions where you can act on them: retry, skip, roll back.
- Map them to **HTTP status codes** in one place (exception handlers / middleware).
- Let unexpected errors bubble up to a global handler that **logs the traceback** and returns a generic 500.
- Use **raise ... from e** to keep the cause; never **except: pass**.

Follow-up they ask next: ”What’s the difference between Exception and BaseException?” (BaseException also covers KeyboardInterrupt and SystemExit, which you normally must not catch).

INTERVIEW SAY THIS IN THE INTERVIEW

“What is a context manager? Write one.”

- An object that guarantees **setup and teardown** around a block: **__enter__** and **__exit__**.
- **__exit__** runs even if the block raises; returning True would swallow the exception.
- Examples: files, locks, DB transactions, **requests.Session**, temporary settings in tests.
- Quickest way: **@contextmanager** with try / yield / finally.

```
class Timer:
    def __enter__(self):
        self.start = time.perf_counter()
        return self
    def __exit__(self, exc_type, exc, tb):
        self.took = time.perf_counter() - self.start
        return False        # don't hide errors
```

Follow-up they ask next: ”How do you manage several resources at once?” (several items in one with statement, or contextlib.ExitStack for a dynamic number).

RECAP REMEMBER THESE

Module 13 in 6 lines

1. Catch the **narrowest** exception you can handle; never **except: pass**.
2. **else** = success path, **finally** = always.
3. Your own **exception family**; **raise ... from e** keeps the cause.
4. **with** guarantees clean-up; **@contextmanager** writes one in 6 lines.
5. **logging**, not print: levels, **getLogger(__name__)**, **log.exception**.
6. Debug with **breakpoint()**; read tracebacks bottom-up.

PRACTICE NOW

Do exercises/ex13_bank_transfer.py: a transfer that rolls back on any error.